

DJANGO QUERYSETS COMPLETE GUIDE

Beginner → Advanced → Expert → Interview Preparation

PART 1: BASIC QUERYSET CONCEPTS

A **QuerySet** in `Entity["software", "Django", "Python web framework"]` represents a collection of database queries.

Example model:

```
class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.IntegerField()
    stock = models.IntegerField()
    category = models.CharField(max_length=50)
    created_at = models.DateTimeField(auto_now_add=True)
```

Access manager:

```
Product.objects
```

PART 2: RETRIEVE SINGLE OBJECT

get()

Returns one object.

```
product = Product.objects.get(id=1)
product = Product.objects.get(name="Laptop")
```

Raises:

```
Product.DoesNotExist
Product.MultipleObjectsReturned
```

Example:

```
try:  
    product = Product.objects.get(id=10)  
except Product.DoesNotExist:  
    print("Not found")
```

first()

Returns first object or None.

```
product = Product.objects.filter(price__gt=500).first()
```

last()

```
product = Product.objects.last()
```

exists()

Check if record exists.

```
exists = Product.objects.filter(name="Laptop").exists()
```

Returns:

```
True / False
```

get_or_create()

```
product, created = Product.objects.get_or_create(  
    name="Phone",
```

```
        defaults={"price":500}  
    )
```

Returns:

```
(product_object, True/False)
```

update_or_create()

```
product, created = Product.objects.update_or_create(  
    name="Phone",  
    defaults={"price":800}  
)
```

If exists → update

If not exists → create

PART 3: RETRIEVE MULTIPLE OBJECTS

all()

```
products = Product.objects.all()
```

SQL:

```
SELECT * FROM product;
```

filter()

AND condition.

```
products = Product.objects.filter(  
    price=100,
```

```
        category="electronics"  
    )
```

SQL:

```
WHERE price=100 AND category='electronics'
```

exclude()

Opposite of filter.

```
products = Product.objects.exclude(category="deleted")
```

SQL:

```
WHERE NOT category='deleted'
```

PART 4: FIELD LOOKUPS

Exact match

```
Product.objects.filter(name="Laptop")
```

Greater than

```
Product.objects.filter(price__gt=100)
```

SQL:

```
WHERE price > 100
```

Greater than equal

```
Product.objects.filter(price__gte=100)
```

Less than

```
Product.objects.filter(price__lt=500)
```

Less than equal

```
Product.objects.filter(price__lte=500)
```

Contains

Case sensitive.

```
Product.objects.filter(name__contains="phone")
```

SQL:

```
LIKE '%phone%'
```

icontains

Case insensitive.

```
Product.objects.filter(name__icontains="phone")
```

startswith

```
Product.objects.filter(name__startswith="Sam")
```

istartswith

```
Product.objects.filter(name__istartswith="sam")
```

endswith

```
Product.objects.filter(name__endswith="Pro")
```

PART 5: ADVANCED LOOKUPS

in lookup

```
Product.objects.filter(id__in=[1,2,3])
```

SQL:

```
WHERE id IN (1,2,3)
```

range

```
Product.objects.filter(price__range=[100,500])
```

SQL:

```
WHERE price BETWEEN 100 AND 500
```

isnull

```
Product.objects.filter(description__isnull=True)
```

SQL:

```
WHERE description IS NULL
```

regex

```
Product.objects.filter(name__regex=r'^[A-Z]')
```

PART 6: DATE LOOKUPS

Model:

```
created_at = models.DateTimeField()
```

year

```
Product.objects.filter(created_at__year=2025)
```

month

```
Product.objects.filter(created_at__month=6)
```

day

```
Product.objects.filter(created_at__day=10)
```

date

```
Product.objects.filter(created_at__date="2025-01-01")
```

week

```
Product.objects.filter(created_at__week=20)
```

PART 7: Q OBJECTS

Used for OR queries.

Import:

```
from django.db.models import Q
```

OR

```
Product.objects.filter(  
    Q(price__gt=500) | Q(category="featured")  
)
```

SQL:

```
WHERE price > 500 OR category='featured'
```

AND

```
Product.objects.filter(  
    Q(price__gt=500) & Q(stock__gt=0)  
)
```

NOT

```
Product.objects.filter(  
    ~Q(category="deleted")  
)
```

PART 8: ORDERING

Ascending:

```
Product.objects.order_by("name")
```

Descending:

```
Product.objects.order_by("-price")
```

Multiple:

```
Product.objects.order_by("category", "-price")
```

SQL:

```
ORDER BY category ASC, price DESC
```

PART 9: LIMITING

First 10

```
Product.objects.all()[:10]
```

Skip 10 take 10

```
Product.objects.all()[10:20]
```

SQL:

```
LIMIT 10 OFFSET 10
```

PART 10: COUNTING

Count all:

```
count = Product.objects.count()
```

SQL:

```
SELECT COUNT(*)
```

PART 11: DISTINCT

Unique values.

```
Product.objects.values("category").distinct()
```

SQL:

```
SELECT DISTINCT category
```

PART 12: AGGREGATION

Import:

```
from django.db.models import Sum, Avg, Max, Min, Count
```

Total:

```
Product.objects.aggregate(Sum("price"))
```

Average:

```
Product.objects.aggregate(Avg("price"))
```

Max:

```
Product.objects.aggregate(Max("price"))
```

Min:

```
Product.objects.aggregate(Min("price"))
```

Count:

```
Product.objects.aggregate(Count("id"))
```

PART 13: ANNOTATE

Add calculated fields.

Products per category.

```
Product.objects.values("category").annotate(  
    total=Count("id")  
)
```

SQL:

```
GROUP BY category
```

Example:

```
Author.objects.annotate(  
    total_books=Count("book")  
)
```

Each author gets:

```
author.total_books
```

PART 14: UPDATE QUERIES

Update one field.

```
Product.objects.filter(id=1).update(price=1000)
```

SQL:

```
UPDATE product SET price=1000
```

Better than:

```
product = Product.objects.get(id=1)  
product.price = 1000  
product.save()
```

PART 15: DELETE QUERIES

Delete single:

```
Product.objects.get(id=1).delete()
```

Delete multiple:

```
Product.objects.filter(stock=0).delete()
```

SQL:

```
DELETE FROM product WHERE stock=0
```

PART 16: BULK OPERATIONS

bulk_create()

Insert many rows.

```
Product.objects.bulk_create([
    Product(name="Phone"),
    Product(name="Laptop"),
    Product(name="TV")
])
```

Fast.

bulk_update()

```
products = Product.objects.all()

for p in products:
    p.stock = 0

Product.objects.bulk_update(
    products,
    ["stock"]
)
```

PART 17: RELATIONSHIP QUERIES

Models:

```
class Author(models.Model):
    name = models.CharField(max_length=100)
```

```
class Book(models.Model):
    title = models.CharField(max_length=100)
    author = models.ForeignKey(Author, on_delete=models.CASCADE)
```

Find books by author.

```
Book.objects.filter(author__name="John")
```

Find author by book title.

```
Author.objects.filter(book__title="Python")
```

Django follows relation automatically.

PART 18: MANY TO MANY QUERIES

```
class Student(models.Model):
    name = models.CharField(max_length=100)

class Course(models.Model):
    title = models.CharField(max_length=100)
    students = models.ManyToManyField(Student)
```

Students in course:

```
Student.objects.filter(
    course__title="Python"
)
```

Courses by student:

```
Course.objects.filter(
    students__name="Ali"
)
```

PART 19: F EXPRESSIONS

Used to compare fields inside DB.

Import:

```
from django.db.models import F
```

Increase stock.

```
Product.objects.filter(id=1).update(  
    stock=F("stock")+1  
)
```

SQL:

```
UPDATE product SET stock=stock+1
```

No Python calculation.

Compare fields:

```
Product.objects.filter(  
    stock__gt=F("price")  
)
```

PART 20: EXISTS SUBQUERY

Import:

```
from django.db.models import Exists, OuterRef
```

Example:

```
recent = Order.objects.filter(  
    product=OuterRef("pk")
```

```
)

Product.objects.annotate(
    has_order=Exists(recent)
)
```

PART 21: SUBQUERY

Get latest order price.

```
from django.db.models import Subquery

latest = Order.objects.filter(
    product=OuterRef("pk")
).values("amount")[:1]

Product.objects.annotate(
    latest_amount=Subquery(latest)
)
```

PART 22: SELECT_RELATED

Used for:

```
ForeignKey
OneToOneField
```

Problem:

```
for book in Book.objects.all():
    print(book.author.name)
```

Queries:

```
101 queries
```

Fix:

```
Book.objects.select_related("author")
```

Uses:

```
JOIN
```

PART 23: PREFETCH_RELATED

Used for:

```
ManyToMany  
Reverse ForeignKey
```

Problem:

```
for course in Course.objects.all():  
    for student in course.students.all():  
        print(student.name)
```

Many queries.

Fix:

```
Course.objects.prefetch_related(  
    "students"  
)
```

Uses:

```
Separate queries + Python combine
```

PART 24: ONLY AND DEFER

Only load selected fields.

```
Product.objects.only("name", "price")
```

Skip large fields.

```
Product.objects.defer(  
    "description",  
    "long_text"  
)
```

PART 25: VALUES

Return dictionary.

```
Product.objects.values("name", "price")
```

Output:

```
[  
    {"name": "Phone", "price": 500}  
]
```

PART 26: VALUES_LIST

Return tuple.

```
Product.objects.values_list(  
    "name",  
    "price"  
)
```

Output:

```
[  
    ("Phone", 500)  
]
```

Flat list:

```
Product.objects.values_list(  
    "name",  
    flat=True  
)
```

Output:

```
["Phone", "TV"]
```

PART 27: RAW SQL

```
Product.objects.raw(  
    "SELECT * FROM product"  
)
```

Using cursor:

```
from django.db import connection  
  
cursor = connection.cursor()  
  
cursor.execute(  
    "SELECT * FROM product"  
)
```

Use parameters.

```
cursor.execute(  
    "SELECT * FROM product WHERE id=%s",  
    [1]  
)
```

Avoid SQL injection.

PART 28: TRANSACTIONS

Import:

```
from django.db import transaction
```

Example:

```
with transaction.atomic():  
  
    order = Order.objects.create(...)  
  
    product.stock -= 1  
    product.save()  
  
    Payment.objects.create(...)
```

All success:

```
COMMIT
```

One error:

```
ROLLBACK
```

PART 29: SELECT_FOR_UPDATE

Row locking.

```
with transaction.atomic():  
  
    account = Account.objects.select_for_update().get(id=1)
```

```
account.balance -= 100
account.save()
```

Prevents race conditions.

PART 30: CUSTOM MANAGERS

```
class PublishedManager(models.Manager):

    def get_queryset(self):
        return super().get_queryset().filter(
            status="published"
        )
```

Model:

```
class Post(models.Model):
    status = models.CharField(max_length=20)

    objects = models.Manager()
    published = PublishedManager()
```

Use:

```
Post.published.all()
```

PART 31: INTERVIEW QUESTIONS

Q1: Difference between get() and filter()

get()

```
Product.objects.get(id=1)
```

Returns:

One object
Raises exception if missing

filter()

```
Product.objects.filter(id=1)
```

Returns:

QuerySet
Empty if missing

Q2: What is QuerySet lazy evaluation?

```
products = Product.objects.filter(price=100)
```

No SQL runs yet.

SQL runs only when:

```
list(products)
for p in products:
    print(p)
```

Q3: Difference between aggregate and annotate?

aggregate()

```
Product.objects.aggregate(
    Avg("price")
)
```

Returns:

Single result

annotate()

```
Category.objects.annotate(  
    total=Count("product")  
)
```

Returns:

Each object gets value

Q4: What is N+1 problem?

Example:

```
for book in Book.objects.all():  
    print(book.author.name)
```

1 query books
100 author queries

Total:

101 queries

Q5: select_related vs prefetch_related

select_related()

```
ForeignKey  
OneToOne  
SQL JOIN  
1 query
```

prefetch_related()

```
ManyToMany  
Reverse ForeignKey  
Separate queries  
Python combines
```

Q6: What is F expression?

Database-level operation.

```
Product.objects.update(  
    stock=F("stock")+1  
)
```

No Python calculation.

Q7: What is Q object?

Dynamic OR queries.

```
Product.objects.filter(  
    Q(price__gt=100) | Q(stock=0)  
)
```

Q8: What is lazy loading?

Django delays SQL execution until needed.

```
products = Product.objects.all()
```

No SQL yet.

Q9: Why use bulk_create?

Insert many rows faster.

```
Product.objects.bulk_create([...])
```

Instead of:

```
for p in products:  
    p.save()
```

Q10: Why use transactions?

Ensures consistency.

```
with transaction.atomic():
```

All success:

```
COMMIT
```

Error:

```
ROLLBACK
```

FINAL EXPERT TOPICS

Study these for senior interviews.

```
Subquery()  
OuterRef()  
Exists()  
Window Functions  
Database Indexing  
Query Optimization  
QuerySet Caching  
Explain Query Plans  
select_for_update()  
bulk_create()  
bulk_update()
```

- Custom Managers
- Custom QuerySets
- Database Transactions
- Raw SQL
- Signals vs Save Override
- Proxy Models
- Database Routers
- Multi Database Queries